



VISION AND LEARNING LAB

GIẢI THUẬT PHÂN LOẠI

TS. Hoàng Quốc Việt



HUNG YEN UNIVERSITY OF TECHNOLOGY AND EDUCATION



K-Nearest Neighbor



Naïve Bayes



Support Vector Machine



Logistic Regression



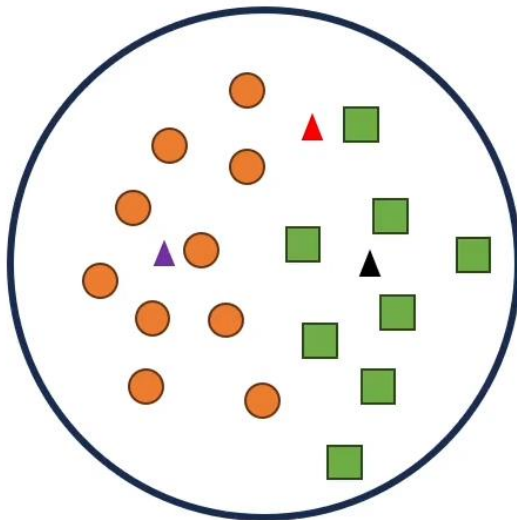
Decision Tree

I. K-Nearest Neighbor

- **K-Nearest Neighbor** (viết tắt KNN): K láng giềng gần nhất.
- KNN: là một trong những thuật toán học máy supervised learning đơn giản.
- KNN có thể áp dụng được vào cả bài toán Classification và Regression.

I. K-Nearest Neighbor

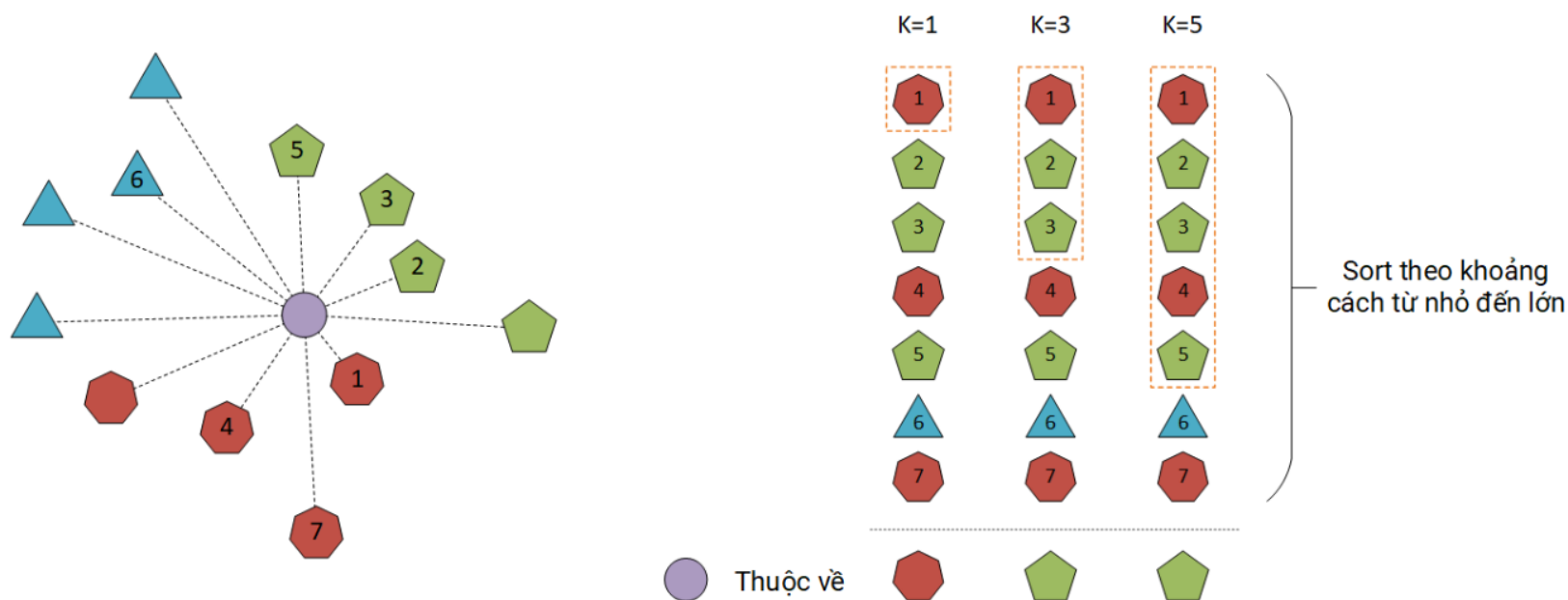
- KNN là thuật toán đi tìm đầu ra của một điểm dữ liệu mới bằng cách dựa trên thông tin của K điểm dữ liệu trong tập dữ liệu huấn luyện gần nó nhất (K-lân cận).



- ▲ tends to be class 1 (green squares)
- ▲ tends to be class 2 (orange circles)
- ▲ depends on **K** value

I. K-Nearest Neighbor

- Trong bài toán phân loại, nhãn của một điểm dữ liệu mới được phân loại trực tiếp từ K điểm dữ liệu gần nhất trong tập training, thường được quyết định bằng việc bầu chọn theo điểm gần nhất nó.



I. K-Nearest Neighbor

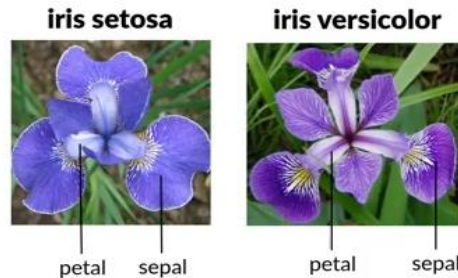


- Giải thuật: Đầu tiên ta có một tập dữ liệu X_{train} . Sau đó, chọn một điểm dữ liệu từ tập X_{test} là x^* :
 - Tính khoảng cách từ x^* tới tất cả các điểm x_i trong tập X_{train} .
 - Sắp xếp các khoảng cách này theo thứ tự tăng dần (từ nhỏ đến lớn).
 - Lấy ra k khoảng cách đầu tiên.
 - Đếm số lần xuất hiện của các label tương ứng với k khoảng cách đầu tiên đó. Label nào xuất hiện nhiều hơn thì chọn.

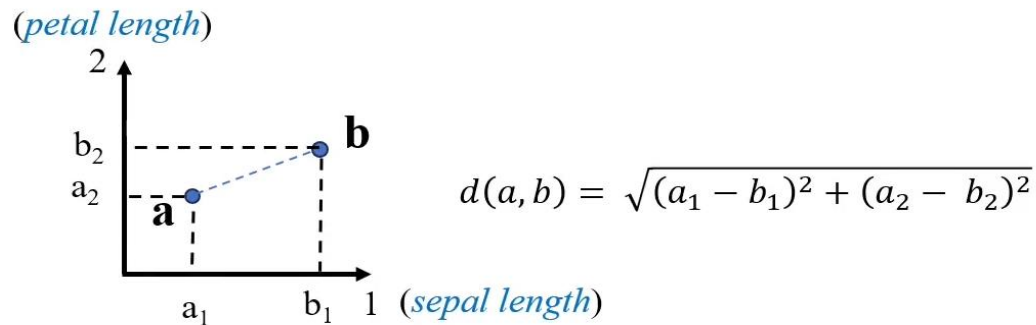


I. K-Nearest Neighbor

- Tính khoảng cách: L2 Norm (Euclidean Distance):

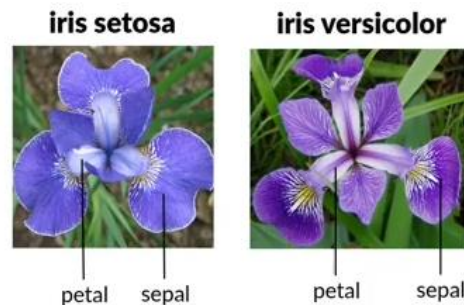


Sepal length	Petal length	classes
5.2	1.4	setosa
6.3	1.5	versicolor



I. K-Nearest Neighbor

- Tính khoảng cách: L2 Norm (Euclidean Distance):



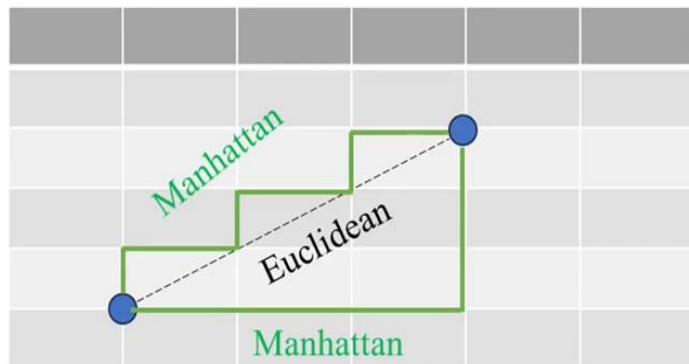
Sepal length	Petal length	classes
5.2	1.4	setosa
6.3	1.5	versicolor

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + (a_3 - b_3)^2}$$

$$d(a, b) = \sqrt{\sum_{i=1}^N (a_i - b_i)^2} \quad \text{Euclidean distance}$$

I. K-Nearest Neighbor

- Tính khoảng cách: L1 Norm (Manhattan Distance):



Manhattan distance

$$d(a, b) = \sum_{i=1}^N |a_i - b_i|$$

Minkowski distance

$$d(a, b) = \left(\sum_{i=1}^N (a_i - b_i)^p \right)^{\frac{1}{p}}$$

Calculate distances



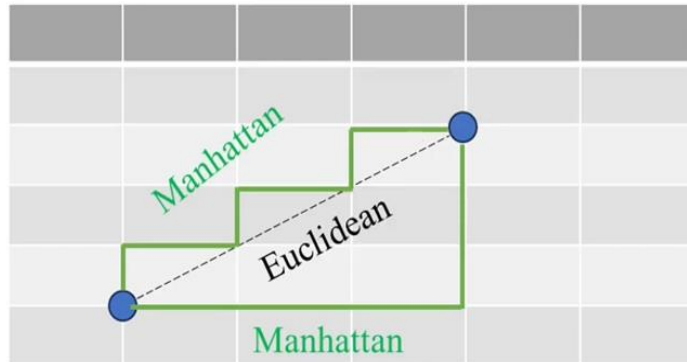
Get K nearest neighbors



Make prediction

I. K-Nearest Neighbor

- Tính khoảng cách: L1 Norm (Manhattan Distance):



Manhattan distance

$$d(a, b) = \sum_{i=1}^N |a_i - b_i|$$

Minkowski distance

$$d(a, b) = \left(\sum_{i=1}^N (a_i - b_i)^p \right)^{\frac{1}{p}}$$

Calculate distances

Get K nearest neighbors

Make prediction

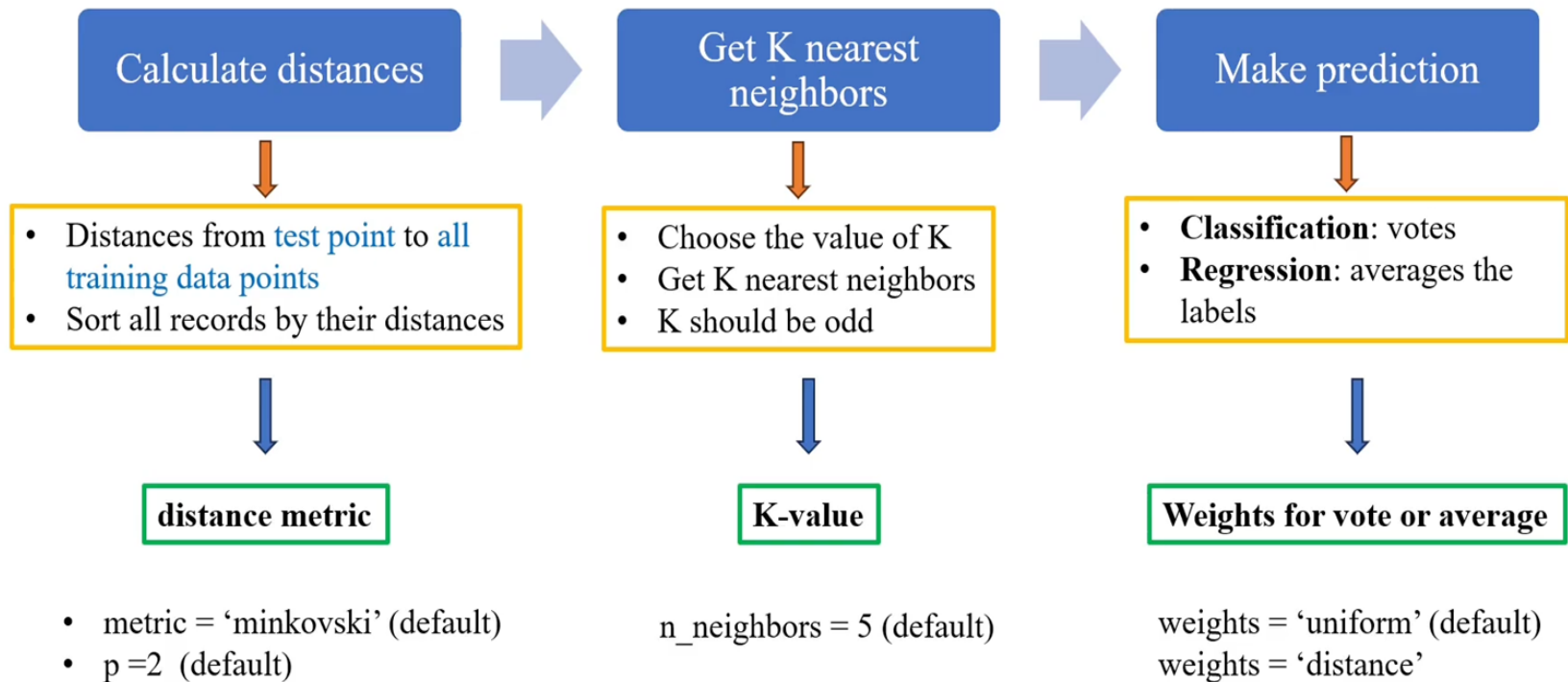
- Distances from **test point** to **all training data points**
- Sort all records by their distances

- Choose the value of K
- Get K nearest neighbors
- K should be odd

- **Classification:** votes
- **Regression:** averages the labels

I. K-Nearest Neighbor

■ Giải thuật:



■ K nhỏ:

- Độ phức tạp cao: Một k nhỏ khiến mô hình tinh chỉnh chi tiết để phù hợp với dữ liệu huấn luyện, dễ dẫn đến overfitting.
- Nhạy cảm với nhiễu: Mô hình sẽ nhạy cảm với nhiễu do ảnh hưởng lớn của các điểm dữ liệu ngoại lai trong quyết định phân loại.

■ K lớn:

- Độ phức tạp thấp: Sử dụng nhiều hàng xóm hơn giúp đường biên quyết định đơn giản và mượt mà hơn, giảm nguy cơ overfitting và cải thiện khả năng tổng quát hóa.
- Chống nhiễu tốt hơn: Ảnh hưởng của nhiễu giảm bớt do quyết định dựa trên nhiều điểm, làm giảm tác động của các điểm cá biệt.

■ Code:

```
# 1. XỬ LÝ DỮ LIỆU
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1.1. Đọc dữ liệu
data = pd.read_csv("Dataset/diabetes.csv")

# 1.2. Tách đặc trưng (X) và nhãn (y)
X = data.iloc[:, :-1]   # tất cả cột trừ cột cuối
y = data.iloc[:, -1]    # cột cuối (label: có/không tiểu đường)

# 1.3. Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.4. Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
# X_test = scaler.transform(X_test)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

I. K-Nearest Neighbor

■ Code:

```
# 2. GIẢI THUẬT KNN
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình KNN
model_knn = KNeighborsClassifier(n_neighbors=5) # k=5
model_knn.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
y_pred = model_knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
```

II. Naïve Bayes

- Định lý Bayes được sử dụng rộng rãi trong thống kê.
- Định lý Bayes được sử dụng để suy luận, ra quyết định, và dự đoán dựa trên xác suất.
- Trong các mô hình Bayes, thông tin ban đầu (tiên nghiệm) được cập nhật dần dần bằng dữ liệu mới (hậu nghiệm), giúp hệ thống đưa ra kết luận chính xác hơn.
- Thường được áp dụng trong Học có giám sát (supervised learning).

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

■ Trong đó :

- $P(A|B)$: Xác suất hậu nghiệm (posterior), xác suất sự kiện A xảy ra nếu B đã xảy ra.
- $P(A)$: Xác suất tiên nghiệm (prior), xác suất của A khi không quan tâm đến B.
- $P(B)$: Xác suất xảy ra của B khi không quan tâm đến A.
- $P(B|A)$: Xác suất có điều kiện, xác suất sự kiện B xảy ra nếu A đã xảy ra.

II. Naïve Bayes

- Naive Bayes là một trong nhóm các thuật toán áp dụng định lý Bayes với một giả định (khá ngây thơ – Naïve): Mọi features đầu vào đều độc lập với nhau.
- Features ở đây là danh sách các biến đầu vào: độ tuổi, giới tính, mức lương, chiều cao...
 - Ví dụ 2 biến độc lập là: Size giày và giới tính của bạn.
 - Ví dụ 2 biến phụ thuộc là: Số tiền quảng cáo bỏ ra và doanh số thu được.

II. Naïve Bayes

- Khi áp dụng định lý Bayes, xác suất của **biến lớp y** dựa trên **vector đặc trưng X** được tính như sau:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

- **Biến lớp y** : Là nhãn mà chúng ta muốn dự đoán (ví dụ: “mưa” hoặc “Không mưa”).
- **Vecto đặc trưng X** : Là tập hợp các thuộc tính của dữ liệu, ví dụ:
$$X = (x_1, x_2, x_3, \dots, x_n)$$
- Ví dụ: Nếu $X=(\text{Ít}, \text{Thấp}, \text{Bắc})$ và $y=\text{Không mưa}$, điều này có nghĩa là xác suất “Không mưa” với điều kiện thời tiết là “Mây ít”, “Áp suất thấp” và “Gió Bắc”.

- Giải thuật:
 - *Giả định độc lập*: Các đặc trưng là độc lập với nhau.
 - *Tính xác suất có điều kiện*: Sử dụng các xác suất có điều kiện của từng đặc trưng để tính xác suất tổng cho lớp.
 - *Dự đoán lớp*: Chọn lớp có xác suất cao nhất cho thể hiện cần phân loại.

- Ví dụ minh họa:



20 Normal

$$P(\text{Normal}) = \frac{20}{30} = 67\%$$

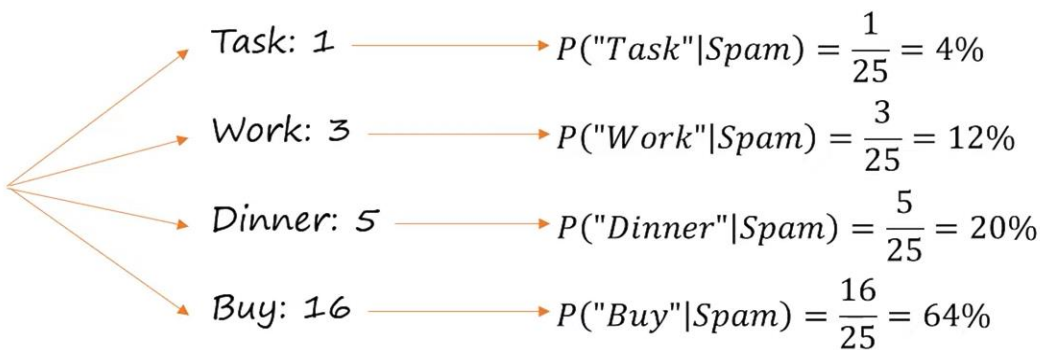
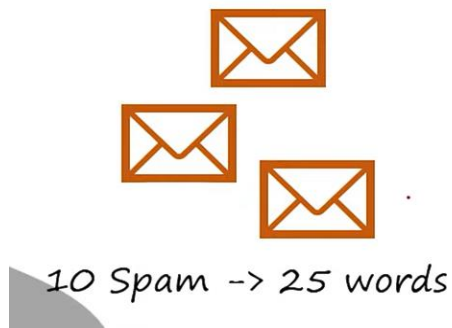
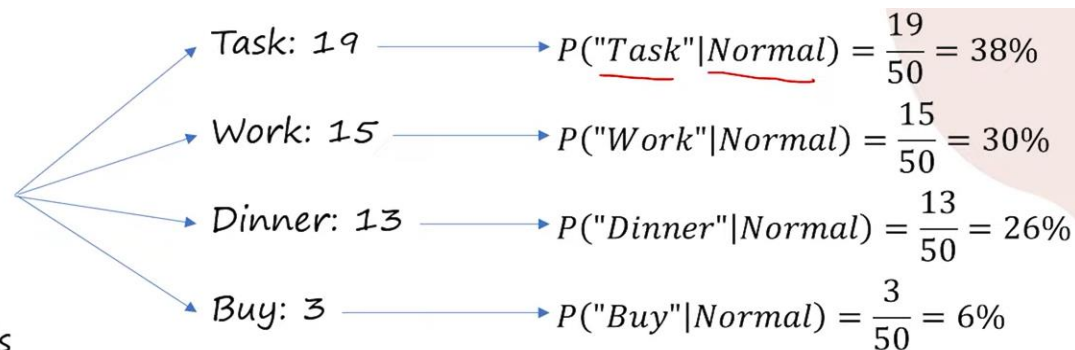


10 Spam

$$P(\text{Spam}) = \frac{10}{30} = 33\%$$

II. Naïve Bayes

■ Ví dụ minh hoạ:



II. Naïve Bayes

■ Ví dụ minh họa:

P(Normal)=67%	P("Task" Normal) = 38%
	P("Work" Normal) = 30%
	P("Dinner" Normal) = 26%
	P("Buy" Normal) = 6%

Giả sử mình nhận được 1 email với nội dung "Task Dinner". Thuật toán sẽ tự động phân loại đây là email *normal* hay *spam*?

P(Spam)=33%	P("Task" Spam) = 4%
	P("Work" Spam) = 12%
	P("Dinner" Spam) = 20%
	P("Buy" Spam) = 64%

- $P(\text{Normal} | \text{"Task Dinner"}) = ?$
- $P(\text{Spam} | \text{"Task Dinner"}) = ?$
- $\Rightarrow$ Thuật toán sẽ phân loại email dựa trên xác suất cao hơn

■ Ví dụ minh họa:

$$\begin{aligned}
 P(\text{Normal} | \text{"Task Dinner"}) &= \frac{P(\text{Normal} \cap \text{"Task Dinner"})}{P(\text{"Task Dinner"})} \\
 &= \frac{P(\text{Normal}) \cdot P(\text{"Task Dinner"} | \text{Normal})}{P(\text{"Task Dinner"})} \\
 &= \frac{P(\text{Normal}) \cdot P(\text{"Task"} | \text{Normal}) \cdot P(\text{"Dinner"} | \text{Normal})}{P(\text{"Task Dinner"})} \\
 &= \frac{0,67 \cdot 0,38 \cdot 0,26}{P(\text{"Task Dinner"})} \\
 &= \frac{0,066}{P(\text{"Task Dinner"})}
 \end{aligned}$$

$$\begin{aligned}
 P(\text{Spam} | \text{"Task Dinner"}) &= \frac{P(\text{Spam} \cap \text{"Task Dinner"})}{P(\text{"Task Dinner"})} \\
 &= \frac{P(\text{Spam}) \cdot P(\text{"Task Dinner"} | \text{Spam})}{P(\text{"Task Dinner"})} \\
 &= \frac{P(\text{Spam}) \cdot P(\text{"Task"} | \text{Spam}) \cdot P(\text{"Dinner"} | \text{Spam})}{P(\text{"Task Dinner"})} \\
 &= \frac{0,33 \cdot 0,04 \cdot 0,2}{P(\text{"Task Dinner"})} \\
 &= \frac{0,0026}{P(\text{"Task Dinner"})}
 \end{aligned}$$

- $P(\text{Normal} | \text{"Task Dinner"}) > P(\text{Spam} | \text{"Task Dinner"}) \Rightarrow$
Email này được phân loại là email Normal.

$$P(\text{"Task Dinner"}) = P(\text{"Task Dinner"} \mid \text{Normal}) \cdot P(\text{Normal}) + \\ P(\text{"Task Dinner"} \mid \text{Spam}) \cdot P(\text{Spam})$$

■ Trong đó:

- $P(\text{"Task Dinner"})$: Xác suất tổng quát 1 tin nhắn chứa "Task Dinner".
- $P(\text{"Task Dinner"} \mid \text{Normal})$: Xác suất cụm từ "Task Dinner" xuất hiện trong 1 tin nhắn Normal.
- $P(\text{Normal})$: Xác suất tổng quát 1 tin nhắn là Spam, giá trị này được ước lượng từ tập dữ liệu (=0,67).
- $P(\text{"Task Dinner"} \mid \text{Spam})$: Xác suất cụm từ "Task Dinner" xuất hiện trong 1 tin nhắn Spam.
- $P(\text{Spam})$: Xác suất tổng quát 1 tin nhắn là Spam, giá trị này được ước lượng từ tập dữ liệu (=0,33). Có thể được tính bằng $P(\text{Spam}) = 1 - P(\text{Normal})$.

■ Naive Bayes:

STT	Weather	Father	Mother	Attention	Result
1	good	stay home	stay home	hate	no
2	normal	stay home	go	very hate	yes
3	very good	go	stay home	like	yes
4	good	go	go	like	no
5	normal	stay home	go	like	yes
6	good	go	go	like	yes
7	bad	go	stay home	like	no
8	good	go	go	hate	no
9	very good	go	stay home	very like	no
10	good	stay home	go	hate	yes



Weather				
	Yes	No	P(yes)	P(no)
good	2	3	2/5	3/5
very good	1	1	1/5	1/5
normal	2	0	2/5	0/5
bad	0	1	0/5	1/5

Father				
	Yes	No	P(yes)	P(no)
Stay home	3	1	3/5	1/5
go	2	4	2/5	4/5

$$P(A) \longrightarrow P(\text{Yes}) = 5/10 = 1/2$$

$$P(\text{No}) = 5/10 = 1/2$$

- $P(\text{weather} = \text{good} \mid \text{result} = \text{yes}) = 2/5$
- $P(\text{weather} = \text{good} \mid \text{result} = \text{no}) = 3/5$

- Thuật toán Naive Bayes có các bộ phân lớp khác nhau, tùy thuộc vào hàm phân phối của các đặc trưng:
 - *Multinomial Navie Bayes*: là hàm phân phối đa thức, thường dùng cho các đặc trưng có giá trị rời rạc.
 - *Gaussian Navie Bayes*: là hàm phân phối Gauss, thường dùng cho các đặc trưng có giá trị liên tục.
 - *Bernoulli Navie Bayes*: là hàm phân phối Bernoulli đa biến, thường dùng với các đặc trưng là các boolean độc lập (biến nhị phân).

II. Naïve Bayes

- Code:
 - Sử dụng mô hình Naïve bayes cho bài toán phân loại.
 - Sử dụng thư viện *sklearn*:

```
# 3. GIẢI THUẬT NAIVE BAYES
from sklearn import naive_bayes
from sklearn.metrics import classification_report, accuracy_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Naive bayes
model_nb = naive_bayes.GaussianNB()
model_nb.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
y_pred_nb = model_nb.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("\nClassification Report:\n", classification_report(y_test,
y_pred_nb))
```

III. Support Vector Machine

- Support Vector Machine:

■ Code:

```
# 3. GIẢI THUẬT SUPPORT VECTOR MACHINE
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Support Vector Machine
model_svm = SVC(kernel='rbf', C=1.0, gamma='scale',
random_state=42)
model_svm.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
y_pred_svm = model_svm.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test,
y_pred_svm))
```

IV. Logistic Regression

- Logistic Regression:

■ Code:

```
# 4. GIẢI THUẬT LOGISTIC REGRESSION
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Logistic Regression
model_lr = LogisticRegression(solver='liblinear',
                              multi_class='ovr')
model_lr.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
y_pred_lr = model_lr.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("\nClassification Report:\n", classification_report(y_test,
y_pred_lr))
```

- Decision Tree

■ Code:

```
# 5. GIẢI THUẬT DECISION TREE
from sklearn import tree
from sklearn.metrics import classification_report, accuracy_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Decision Tree
model_dt = tree.DecisionTreeClassifier(criterion='gini')
model_dt.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
y_pred_dt = model_dt.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test,
y_pred_dt))
```

- [1]. <https://medium.com/>
- [2]. <https://towardsdatascience.com/>
- [3]. <https://machinelearningcoban.com/>
- [4]. Huang, Shih-Chia, and Trung-Hieu Le. *Principles and labs for deep learning*. Academic Press, 2021.

Thank You !

Question?